



Attestium

Attestium: A Framework for Verifiable Runtime Integrity

An open-source framework for verifiable,
hardware-backed runtime integrity

Forward Email LLC
<https://forwardemail.net>
<https://github.com/attestium/attestium.com>

Foreword

Supply chain attacks such as the SolarWinds compromise and the xz-utils backdoor exposed the gap between build-time security and runtime integrity. Frameworks such as SLSA and tools such as Sigstore secure software up to the point of release; they do not show what a server runs after deployment.

This whitepaper describes Attestium, an open-source library and evidence format for remote attestation of what a server runs. An attester on the server reports its files, installed packages, running processes, executables, libraries and containers; a verifier elsewhere compares each of them with a reference the operator does not control, such as the public commit, the artifacts a lockfile pins, a container image, an official release or a signed distribution archive, and reports any running code that no reference explains. The approach applies to any language and any binary. Where a TPM or a confidential virtual machine is available, the evidence is bound to hardware-signed statements, and every result states which level of evidence it reached. The result is continuous, independent verification that the code running in production is the code that was published.

Contents

1	Introduction	3
2	The Problem	4
2.1	Build-Time Security is Not Enough	4
2.2	Where Existing Solutions Fall Short	4
3	The Solution	5
3.1	Components	6
3.2	The Stack	7
3.3	Why GitHub Actions	7
3.4	The Verification Flow	7
4	Systems Architecture	8
4.1	Roles and Trust Boundaries	8
4.2	Evidence Format	9
4.3	Services and Processes	10
4.4	Explaining Every Executable and Library	11
4.5	Package Ecosystems	12
4.6	Releases, Attestations and Signed Checksums	13
4.7	Containers	14
4.8	Hardware Binding	14
4.9	Monitoring Between Audits	16
4.10	Audit Status: Attester, Verifier and Transports	16
4.11	The Core Library API	17
5	Security Architecture	17
5.1	Cryptographic Primitives	17
5.2	Evidence Levels	18
5.3	Forged Answers	19
5.4	What Each Reference Proves	21
5.5	What the Verifier Refuses	22
5.6	Process Memory Integrity	22
5.7	Threat Model	23
6	Our Security Background	24
6.1	Ansible-Managed Infrastructure	24
6.2	Audits and Continuous Verification	26
6.3	From Hardening to Verification	27
7	The Attestium Stack in Context	28
7.1	Related Work	28
7.2	Architecture Overview	29
7.3	Implementation	29
8	Roadmap	30
8.1	Implemented	30
8.2	Remaining Work	31

8.3	Community and Collaboration	32
9	Adoption	32
9.1	Reference Adopter: Forward Email	32
9.2	How Other Services Adopt	33
9.3	A Call to the Open-Source Community	34
9.4	Preventing the Insider Threat	35
10	Conclusion	36
	References	36

1 Introduction

Attestium is an open-source library and an open evidence format for **remote attestation of what a server runs**. Its purpose is narrow: to let a third party check, without trusting the operator, that a service runs the open-source code it has published. The check covers the service’s own files, every package it installs from a registry, the language runtime that executes it, the executables and shared libraries its processes map, and the containers it runs in. It applies to any language and to any binary, because every file that runs is either explained by a public reference the operator does not control or reported as unexplained.

The SolarWinds [1] and xz-utils [2] compromises showed how much damage a change to shipped code can do before anyone notices. The industry has responded mainly at build time, with signed artifacts, provenance and hardened build pipelines [3], [4]. These are necessary, but they end at deployment. Once code is on a server, a user of the service has no way to know whether it is the code that was built, reviewed and published. Sonatype’s annual report counted more than 245,000 malicious open-source packages in a single year [5], and nothing in a typical deployment checks the installed files against what a lockfile pinned.

System Transparency, developed for Mullvad’s VPN servers, showed that verifiable infrastructure is possible at the level of the boot chain [6], [7]. Attestium applies the same idea to the running application: the files it loads, the executable bytes mapped into its processes, and the upstream releases its runtime and dependencies came from. Executive Order 14028 [8] and the NIST Secure Software Development Framework [9] both call for stronger assurance about the software in use; Attestium is a building block for that assurance.

The design follows the IETF Remote ATtestation procedureS (RATS) architecture [10]. An **attester** on the audited machine collects facts and never judges them. A **verifier** elsewhere sends a fresh nonce, receives the evidence, and compares every fact with a reference it obtains itself: a public commit, a reproduced build, a registry package pinned by a lockfile, a container image by digest, an official runtime release, a signed checksum list, or a package from a distribution’s signed archive. **Relying parties**, such as the users of a service, read the verifier’s results. Where the machine has a TPM 2.0 [11] or runs as a confidential virtual machine (AMD SEV-SNP [12] or Intel TDX [13]), the evidence is bound to hardware-signed statements, and each result states which level of evidence it reached.

Audit Status is a separate project built on Attestium: a ready-made attester and verifier, with deployment tooling for servers and Kubernetes clusters and a GitHub Action that publishes results. Attestium defines what evidence is and how each part of it is checked; Audit Status runs those checks for a fleet and publishes the outcome. Forward Email uses both to publish, on its status page, whether its production servers run the code in its public repository.

The rest of this paper is organized as follows. Section 2 describes the gaps in existing tools. Section 3 states the approach, and Section 4 the architecture: the evidence format, the chain that explains every running executable and library, package ecosystems, containers, hardware binding and transports. Section 5 states what each level of evidence proves and does not prove, and the threat model. Section 6 describes the operational background the project came from, Section 7 places it among related work and describes its implementation, Section 8 lists what is implemented and what remains, and Section 9 describes adoption.

2 The Problem

Most software supply chain security work secures software before it is deployed. What happens after deployment is largely unchecked, and that is where a user’s trust in a service actually rests.

2.1 Build-Time Security is Not Enough

SLSA [3], in-toto [14] and Sigstore [4], [15] create a verifiable chain from source code to a signed artifact. That chain ends when the artifact is deployed. Google’s description of Binary Authorization for Borg makes the same point: alongside checks at deploy time, it continuously re-validates what is already running [16]. A service that publishes its source code gives its users something to read, but not a way to confirm that the published code is what answers their requests.

2.2 Where Existing Solutions Fall Short

A survey of existing verification, attestation and integrity monitoring tools found nine gaps that none of them closed:

1. **Runtime Application Verification Gap:** Most tools verify at build time, at deployment time, or at the level of the boot chain. They do not check what an application is running now.
2. **Third-Party Verification Gap:** Few tools produce evidence that an outside party can check independently. Most report a verdict computed on the machine being checked, which is only as trustworthy as that machine.
3. **Explanation Gap:** File integrity tools compare files with an earlier snapshot of the same machine. A snapshot taken after a compromise is a bad baseline. What is needed is a reference the operator does not control for every file that runs, and an explicit report of every file that has none.
4. **Language and Binary Coverage Gap:** A real service runs an interpreter or a compiled binary, libraries from its operating system distribution, packages from one or more language registries, and often containers. Tools that cover one ecosystem leave the rest unchecked.

5. **Hardware Binding Gap:** Hardware roots of trust (TPMs, confidential virtual machines) prove which machine produced a statement and how it booted, but they say nothing about the application unless its evidence is bound into the hardware-signed statement.
6. **Continuous Verification Gap:** A point-in-time audit cannot detect a change that was made and undone between two audits.
7. **Process Memory Integrity Gap:** File checks verify files on disk, not what is running in memory. An attacker who injects code with `ptrace`, writes to `/proc/<pid>/mem`, or loads a library with `LD_PRELOAD` passes every file-level check because the file on disk is unchanged [17]. Each language runtime adds its own injection paths: `NODE_OPTIONS`, `PYTHONPATH`, Java agents, `RUBYOPT`, `.NET` start-up hooks and debug ports.
8. **Supply Chain Provenance Gap:** A package in a registry can differ from its public source, and a package on disk can differ from what the registry served. Provenance attestations help for packages that publish them [18], but most packages do not, and nothing in a typical deployment checks the installed files against the exact artifacts the lockfile pinned.
9. **Cost and Complexity Gap:** Hardware-based attestation frameworks require specialized knowledge and infrastructure. Small teams need something that installs as one binary and runs in the CI system they already use.

These gaps motivate the approach described in the next section.

3 The Solution

Attestium is a set of small modules that together produce and appraise evidence of what a machine runs. It is organized around six principles.

1. **The attester reports; the verifier judges.** Nothing collected on the audited machine includes a verdict. The machine reports hashes, paths, process state and hardware statements; pass and fail are decided elsewhere.
2. **Every fact is compared with a reference the operator does not control.** A deployed file is compared with the public commit, a reproduced build, or an attested release. An installed package is compared with the registry artifact whose hash the lockfile at that commit pins. A running binary is compared with an official release, a signed checksum list, a container image fetched by digest, or the package that owns it in the distribution's signed archive.
3. **Every running executable and library must be explained.** The verifier does not stop at the application's directory. Every file that an inspected process runs or maps executable is matched with a reference, and a file that no reference explains is reported.

4. **Hardware binds the evidence when it is available.** A TPM quote or a confidential VM report binds the verifier's nonce and a digest of the whole evidence. Each result states which level of evidence it reached (Section 5).
5. **Incomplete is never passing.** A check that could not run, a reference that could not be fetched, or a package whose reference cannot exist makes the result inconclusive or unverifiable, never passing.
6. **The format is open.** Evidence follows a published specification and JSON Schema, independent of language and transport, so attesters and verifiers written in other languages interoperate.

3.1 Components

- **Evidence format, version 2:** A JSON document with services, processes, executables, libraries, installed packages, an optional record of what ran between audits, and optional TPM, IMA and confidential VM statements, all covered by a canonical digest (Section 4.2).
- **Process integrity and runtime profiles:** The executable pages of every file-backed mapping compared byte for byte with the file, memory map anomalies, dynamic linker injection, tracers, `memfd` payloads and listening sockets, plus the code-injection vectors and debug ports of Node.js, Python, the JVM, Ruby, .NET, Erlang/Elixir, PHP, Perl, Deno and Bun.
- **Package ecosystems:** Installed packages of npm, PyPI, RubyGems, Hex, Composer, Maven and NuGet compared file by file with the artifacts their lockfiles pin; the dependencies compiled into Go and Rust binaries compared with `go.sum` and `Cargo.lock`.
- **Release references:** Official Node.js releases, release manifests with GitHub artifact attestations [19], npm provenance, Sigstore bundle verification with a TUF client for Sigstore's trust root [20], and checksum lists signed with OpenPGP, minisign or Sigstore.
- **Operating system packages:** The owner of each executable and library in the `dpkg` database, checked against the Debian or Ubuntu archive through its signed `InRelease` file [21].
- **Containers:** The image, mounts, writable layer and root filesystem of each container, compared with the image fetched from its registry by digest [22].
- **Hardware attestation:** TPM 2.0 quotes and endorsement key enrollment, replay of the Linux IMA measurement log [23], and AMD SEV-SNP and Intel TDX reports obtained through `configs-asm` [24], all verified in pure JavaScript on the verifier.
- **Monitor:** An eBPF program that records every program executed, and every file mapped executable, between two audits.

3.2 The Stack

The system has three parts, each with a separate purpose:

1. **Attestium (library and format):** The evidence format and the primitives to collect and appraise each part of it. It makes no decisions about policy and runs no network service.
2. **Audit Status (attester and verifier):** A single executable built on Attestium. On a server it collects evidence (`auditstatus ssh` behind a restricted SSH key, or `auditstatus serve` on the loopback interface of a Kubernetes pod). Elsewhere it verifies that evidence against public references (`auditstatus verify`) and writes reports and a badge.
3. **Publication:** A scheduled job, typically a GitHub Actions workflow in a public repository, runs the verifier, commits its reports, opens an issue when a server does not pass, and serves a badge that a status page displays.

3.3 Why GitHub Actions

Running the verifier in a public repository's Actions adds no new processor to a service's data pipeline: most open-source projects already trust GitHub with their source code and CI. The workflow definition, its logs and every committed result are public, so anyone can read what was checked and when, and can rerun the same verifier against the same references.

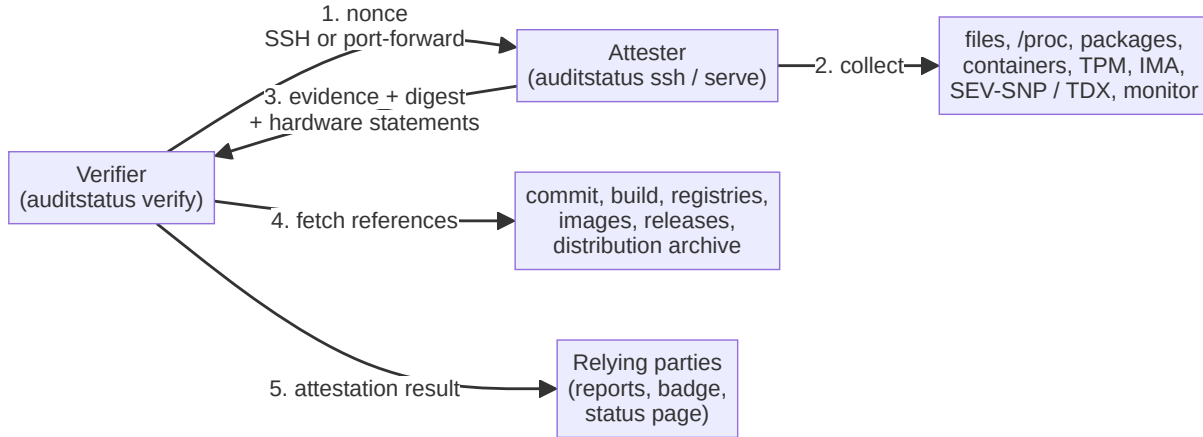
The pattern is not tied to GitHub. `auditstatus verify` is a stateless command that reads a configuration file and writes `report.json`, `report.md` and a Shields.io endpoint badge. It runs the same way in any CI system or on a separate host. The Audit Status GitHub Action wraps it with optional publication to a branch, an issue that opens when the audit stops passing and closes when it passes again, and a configurable failure threshold.

3.4 The Verification Flow

The flow follows the RATS roles [10]. The verifier generates a random nonce and asks the attester for evidence over one of two transports: an SSH key whose `authorized_keys` entry forces the attester command, or a Kubernetes port-forward to an attester that listens only on its pod's loopback interface. The attester collects evidence, computes its digest, and asks the hardware to bind the nonce and the digest:

- TPM 2.0 quote, qualifying data: `SHA-256(nonce || evidenceDigest)`
- Confidential VM report, report data: `SHA-512(nonce || evidenceDigest)`

The verifier checks the evidence against the schema, the nonce, the time window and the recomputed digest, verifies the hardware statements with keys it pinned or vendor roots it ships, and appraises every fact against references it fetches itself.



The server never decides whether it passed. It reports what it sees; the verdict comes from another machine, using references the server’s operator does not control.

4 Systems Architecture

This section describes the roles, the evidence format, and each kind of reference the verifier uses, then the Audit Status attester and verifier that run them.

4.1 Roles and Trust Boundaries

The roles are those of RFC 9334 [10]:

- **Attester:** runs on the audited machine. It collects facts and binds them to the hardware when possible. It is trusted for nothing beyond what the hardware backs (Section 5).
- **Verifier:** runs on a separate machine, usually in CI. It chooses the nonce, checks the evidence, and obtains every reference itself, by digest wherever the reference is content-addressed.
- **Reference sources:** public repositories, package registries, container registries, official release sites, and distribution archives. The verifier authenticates each through hashes pinned by something it already trusts (a lockfile at a public commit, a signed archive index, a Sigstore bundle, a vendor certificate chain).
- **Relying parties:** read the verifier’s attestation results.

In RATS terms, Attestium evidence is RATS evidence and a verifier’s report is an attestation result. The verifier follows the background-check model: the relying party trusts the verifier, and the verifier holds the reference values and endorsements (pinned attestation keys, vendor roots, expected measurements).

4.2 Evidence Format

Evidence version 2 is specified in the repository’s `SPEC.md` and published as a JSON Schema (draft 2020-12). The verifier validates evidence against the schema before reading any field; the validator implements exactly the keywords the schema uses and rejects unknown keywords, so the schema cannot silently skip a check. Objects the schema closes reject unknown fields, and the version changes whenever a change could make a verifier misread evidence.

The top level holds:

Field	Content
<code>nonce, collectedAt</code>	The verifier’s nonce (16 to 64 bytes, hex) and the collection time
<code>attester</code>	Name, version, platform and the SHA-256 of the attester’s own executable
<code>host</code>	Hostname, kernel, boot id, and operating system release
<code>services</code>	Directory services and container services (Section 4.3)
<code>executables, libraries</code>	Every file an inspected process runs or maps executable, once each, with its hash, owning distribution package, and embedded Go or Rust build information
<code>globalPackages</code>	Tools installed next to the Node.js runtime (npm, corepack, pnpm, pm2)
<code>monitor</code>	Programs executed and files mapped executable since the previous window (Section 4.9)
<code>evidenceDigest</code>	SHA-256 of the canonical encoding of everything above
<code>tpm, confidential, ima</code>	Hardware statements, added after the digest

4.2.1 Canonical Digest

`evidenceDigest` is the SHA-256 of the canonical JSON encoding of the evidence without the fields `evidenceDigest`, `tpm`, `ima` and `confidential`. The encoding writes numbers and strings as ECMAScript’s `JSON.stringify` does, omits members whose value is undefined, sorts object keys by UTF-16 code units, and uses no whitespace. Any implementation that follows these rules computes the same digest for the same evidence.

4.2.2 Freshness and Binding

1. The verifier chooses a random nonce and sends it hex-encoded.
2. The attester collects everything, then computes `evidenceDigest`.
3. When hardware is available, the attester binds both values into hardware-signed statements. A TPM 2.0 quote carries the qualifying data `SHA-256(nonce || evidenceDigest)`. A confidential VM report carries the 64 bytes of report data `SHA-512(nonce || evidenceDigest)`. Both concatenate raw bytes, not hex.
4. The IMA log is read after the quote and authenticated by replaying it to the quoted PCR 10 in the SHA-256 bank.

The verifier accepts evidence only if the nonce is its own, `collectedAt` is within its window (15 minutes by default), and the digest it recomputes equals `evidenceDigest`. Because the hardware statement signs the digest, and the digest covers every fact, a quote or report cannot be moved to other evidence or replayed for another nonce. Evidence also records whether the attester was configured to require each kind of hardware, so a missing statement from a machine that should have one fails instead of passing at a lower level.

4.3 Services and Processes

4.3.1 Directory Services

A directory service is a deployed tree, usually a git checkout. The attester reads the commit and ref from `.git` without running git, and records every file (except `.git` and package install directories) with its SHA-256 and git mode; symbolic links are recorded, never followed. A release deployed without git carries a release manifest instead (Section 4.6). The attester also scans installed packages of every detected ecosystem (Section 4.5) and inspects every process whose working directory is inside the root, and it lists the same user’s other processes.

The verifier keeps a blobless clone of the public repository and checks out the reported commit into a worktree, so `.gitattributes` behave as on a normal checkout. It requires the commit to be on the audited branch and recent enough, compares every tracked file’s hash and mode, fails on untracked files that the commit’s `.gitignore` does not ignore, and fails on files the attester could not read unless the commit ignores them.

Deployments often generate files that the repository ignores, such as bundled browser code or a generated parser. When configured, the verifier runs the build in a clean checkout of the same commit, with a minimal environment, and compares every generated file with the server’s copy. The build runs only for commits on the audited branch, so a server cannot make the verifier run arbitrary code, and the build must be reproducible [25].

4.3.2 Processes and Runtimes

For each inspected process the attester reports, from `procf`s:

- **Executable pages:** every executable, file-backed mapping (the program and each shared library) read from `/proc/<pid>/mem` and compared byte for byte with the same range of the file, read through `/proc/<pid>/root` so that processes in containers and chroots are compared with their own view of the filesystem. A single modified byte is reported with its offset. Mappings whose file was replaced since the process started cannot be compared and are reported as such.
- **Memory maps:** executable `memfd` mappings, writable and executable file mappings, deleted or replaced backing files, and libraries outside an optional allowlist. Anonymous executable

memory is counted, because JIT compilers create it legitimately.

- **Dynamic linker:** LD_PRELOAD, LD_AUDIT and /etc/ld.so.preload.
- **Tracer, file descriptors and sockets:** an attached debugger, open memfd objects and deleted files, and listening ports, including a debugger or inspector opened at runtime.
- **Runtime:** the language runtime, detected from the executable name and the libraries mapped into the process (for example libjvm.so identifies a JVM whatever its launcher is called), and that runtime's code-injection vectors.
- **Files changed after start:** files of the service whose contents or status changed after the process started, which indicates a pending restart or code that was loaded and then restored.

Runtime profiles exist for Node.js, Python, the JVM, Ruby, .NET, Erlang/Elixir (BEAM), PHP, Perl, Deno and Bun; any other process is treated as native code. Each profile names the environment variables that load code (NODE_OPTIONS, PYTHONPATH, JAVA_TOOL_OPTIONS, RUBYOPT, DOTNET_STARTUP_HOOKS, ERL_FLAGS, PHP_INI_SCAN_DIR, PERL5OPT and others), the command-line options that do the same, debugger and management ports, and attach mechanisms. For Node.js processes managed by PM2, the options PM2 passed are read from its environment, and a command line overwritten with process.title, which can hide the options a process started with, is recognized.

Findings carry a severity: `critical` is evidence of tampering, `warning` deserves attention, `info` is context. A process with any check in `incomplete` (for example for lack of permission) cannot pass. Linux is fully supported; macOS and Windows support the subset their tooling exposes, and unsupported checks report `supported: false` rather than passing.

4.4 Explaining Every Executable and Library

The evidence lists every file that an inspected process runs or maps executable, once per host or container, with its SHA-256 read through the process. The verifier explains each one by the first reference that matches:

1. **Service files:** a file of a service that matched its commit, reproduced build or attested release.
2. **Packages:** a file of an installed package that matched its locked artifact.
3. **Image:** a file of a container's image, fetched by digest.
4. **Official runtime release:** for Node.js binaries, whose release URL is embedded in the binary, the node binary inside the official archive listed in SHASUMS256.txt, optionally with the release signature checked against the Node.js release keys.
5. **Pinned hash or signed checksum list:** a hash in the verifier's configuration, or a project's published checksum list (SHA256SUMS, checksums.txt) whose OpenPGP, minisign or Sigstore signature verifies.
6. **Distribution package:** the file of the Debian or Ubuntu package that owns it according to the dpkg database, taken from the distribution's archive.

A file that matches no reference is **unexplained** (a warning by default, configurable to fail). A file whose own reference says otherwise, such as a binary that claims to be an official Node.js release but differs from it, or a library that differs from the package that owns it, **fails**.

The distribution reference follows the archive’s chain of trust [21]. The verifier checks the **InRelease** signature with the distribution’s keyring using **gpgv**, follows the SHA-256 in **InRelease** to the **Packages** index and from there to the **.deb**, extracts it, and compares the file. The **dpkg** database is only the server’s claim about which package owns a file; a false claim fails, because the named package must contain the same bytes. A version that the archive no longer carries is looked up in the archive’s snapshot service at the time the package was installed. The archive indexes may be fetched over plain HTTP, because their contents are authenticated by signature and hash rather than by transport.

Compiled Go and Rust binaries also carry the list of modules or crates built into them. Go writes its module list with **go.sum** hashes and its build settings (VCS revision, whether the tree was modified, **-trimpath**) into every binary; binaries built with cargo-auditable carry their crate list. The verifier compares these lists with **go.sum** or **Cargo.lock** at the deployed commit and flags a build from another commit or from a modified tree. This metadata is written by the build and proves nothing about the binary on its own: the binary’s hash, matched against a reproduced build, an attested artifact or a signed checksum, is what proves it. The metadata shows which dependencies went into it.

4.5 Package Ecosystems

Each ecosystem is a plugin with an attester half (find install directories and hash what is there) and a verifier half (read the lockfile at the public commit, fetch the artifacts it pins, and compare). The verifier always reads the lockfile from its own checkout of the public repository, never from the server.

Ecosystem	Installed in	Reference
npm	<code>node_modules</code> (npm and pnpm layouts)	Registry tarball whose SRI integrity <code>package-lock.json</code> or <code>pnpm-lock.yaml</code> pins; GitHub archive for a dependency pinned to a commit
PyPI	virtual environments (<code>site-packages</code> and scripts)	The wheel whose SHA-256 <code>uv.lock</code> , <code>pylock.toml</code> , <code>poetry.lock</code> , <code>Pipfile.lock</code> or a hashed requirements file pins, compared through its <code>RECORD</code> and the wheel install rules
RubyGems	Bundler’s <code>vendor/bundle</code> , <code>GEM_HOME</code>	The <code>.gem</code> whose SHA-256 the <code>CHECKSUMS</code> section of <code>Gemfile.lock</code> pins
Hex	Mix’s <code>deps/</code>	The tarball whose outer checksum <code>mix.lock</code> pins
Composer	<code>vendor/</code>	The git tree of the commit <code>composer.lock</code> pins, minus <code>export-ignore</code> files
Maven	directories of jars	The jar hash that Gradle’s dependency verification metadata or a Maven lockfile pins; otherwise Maven Central by coordinates

Ecosystem	Installed in	Reference
NuGet	published .NET applications	The <code>.nupkg</code> whose content hash <code>packages.lock.json</code> pins
Go	inside the binary	<code>go.sum</code> at the commit
Cargo	inside the binary (cargo-auditable)	<code>Cargo.lock</code> at the commit

Every comparison reports each package as verified, failed (it differs from its reference), unverifiable (no reference can exist, for example a package built on the server from a source distribution, or a dependency from an unpinned source), or error (the reference could not be fetched). Files that belong to no package are listed, because some runtimes execute them: Python runs `.pth` files and `sitecustomize.py` at start-up, and RubyGems loads gem specifications, which are Ruby code. Gem specifications must therefore consist only of literal values and agree with the gem’s own metadata, and executable wrappers must match the template RubyGems generates.

Package managers make a few expected changes during installation, and each is accounted for explicitly rather than ignored: pnpm patches must produce the installed files exactly, packages allowed to run install scripts may add their build output, npm and pnpm rewrite a `bin` file’s CRLF shebang line, and Python bytecode caches are listed rather than hashed. Dependencies bundled inside an npm package are matched against the bundling package’s tarball.

4.6 Releases, Attestations and Signed Checksums

A release deployed without git, such as a compiled binary or a bundle, carries `.attestium-manifest.json` at its root: the repository, the commit, and every file’s SHA-256 and mode. A CI workflow writes the manifest from the build output and attests its SHA-256 with a GitHub artifact attestation [19]. The verifier checks the attestation’s signer (repository, workflow file and ref), that the attestation names the commit the manifest names, that the commit is on the audited branch, and every file against the manifest.

Attestations are Sigstore bundles [15] holding a DSSE envelope with an in-toto statement [14]. A bundle is accepted only when:

1. the signing certificate chains to a certificate authority in Sigstore’s trusted root and was valid when the transparency log entry was made;
2. the transparency log entry is authentic (a signed entry timestamp, or an inclusion proof to a signed checkpoint) and matches this signature, certificate and payload;
3. the DSSE signature over the statement verifies with the certificate;
4. the certificate’s identity (the workflow that signed, its issuer, repository, commit and ref) is the one the verifier requires; and
5. the statement names the artifact’s digest as a subject.

Sigstore’s trusted root and npm’s registry keys are obtained with a TUF client [20] that implements

root rotation, expiry, rollback, threshold, length and hash checks from an initial root shipped with the library. The same machinery verifies npm provenance, which links a package tarball to the repository, commit and workflow that published it, and container image attestations stored as OCI referencers.

For binaries whose projects publish a checksum list, the list is accepted when its signature verifies: a detached OpenPGP signature checked with `gpgv` and a configured keyring, a minisign (Ed25519) signature, or a Sigstore bundle with a required identity. An unsigned list is trusted only as far as HTTPS and its host, and the result says so.

4.7 Containers

For a process in a container, the attester identifies the container from the process’s cgroup (Docker, containerd, CRI-O, Podman, Kubernetes), asks the runtime which image it started from (the Docker Engine API, or `crictl` for CRI runtimes), lists the mounts that bring in files from outside the image, walks the overlay’s writable layer including deletions, and hashes every file of the root filesystem through `/proc/<pid>/root`.

The runtime’s claims about the image are not trusted. The verifier fetches the image from its registry by digest [22]; manifests and layers are content-addressed, so every byte is checked against the digest that names it. It applies the layers in order, including whiteouts, to obtain the image’s root filesystem as a map of file hashes, and compares it with the container’s files, except the few files every runtime writes (such as `/etc/hosts`) and mounted paths. Optionally the image must be attested by a configured GitHub workflow, through a Sigstore bundle stored as an OCI referrer or in GitHub’s attestation store. Volumes are listed, because code in them is outside the image. Image files are recorded as explained, and the container’s processes are judged like any other.

4.8 Hardware Binding

4.8.1 TPM 2.0

The attester uses `tpm2-tools` to create an Attestation Key (AK) under the Endorsement Key (EK), make it persistent, and produce quotes over selected PCRs with the verifier’s qualifying data [11]. The verifier needs no TPM software: it parses `TPMS_ATTEST` and checks the signature (RSA or ECDSA) against the pinned AK, the structure’s magic and type, the qualifying data, the digest of the reported PCR values, and any expected PCR values.

A verifier trusts an AK only after enrollment, which Audit Status performs in three steps:

1. the EK certificate, stored in the TPM by its manufacturer, must chain to a TPM manufacturer’s CA that the verifier trusts and must certify the TPM’s EK public key;
2. the AK’s public area must describe a restricted signing key that cannot leave the TPM (`fixedTPM, fixedParent, sensitiveDataOrigin`);

3. the verifier encrypts a random secret to the EK, bound to the AK's name (**MakeCredential**), and the server must return it; only the TPM holding both keys can decrypt it (**ActivateCredential**).

The resulting AK public key is pinned in the verifier configuration. A TPM without an EK certificate, typically a virtual TPM, is enrolled only when the operator explicitly allows it; credential activation still binds the AK to that EK, but nothing then shows that the TPM is genuine, and enrollment says so. Results note whether the pinned key was enrolled against an endorsement certificate.

4.8.2 IMA

With the kernel's Integrity Measurement Architecture enabled [23], [26], the kernel hashes files as they are executed, mapped executable, or read (according to its policy), appends each measurement to a log, and extends the hash into PCR 10 before the file is used. The verifier parses the binary log (templates `ima-ng`, `ima-sig` and `ima-buf`) and replays it entry by entry until it reaches the quoted PCR 10 value. Only the SHA-256 bank is used, and only when the signed quote selected PCR 10 in it, because in the SHA-1 bank entries of unknown templates replay from digests the log itself supplies. Entries in the replayed prefix are backed by the TPM; entries logged after the quote are not. The verifier compares the kernel's measurements of the service's files with the public commit, and warns when the kernel also measured other contents for the same file since boot.

4.8.3 Confidential Virtual Machines

In a confidential VM the CPU measures the VM's initial memory at launch and signs a report that includes 64 bytes chosen by the guest; the host operator cannot read the guest's memory or forge the report [12], [13]. The attester obtains the report through Linux `configs-asm` [24] with `SHA-512(nonce || evidenceDigest)` as report data, together with the certificates the host provides.

- **AMD SEV-SNP**: the report is signed (ECDSA P-384) by the chip's VCEK or a VLEK. The verifier checks the certificate up to AMD's ARK for the product (Milan, Genoa, Turin), through the ASK for a VCEK or the ASVK for a VLEK, all shipped with the library; requires the certificate's TCB values to match the report's; and rejects a guest policy that allows debugging.
- **Intel TDX**: the quote is signed by the quoting enclave's attestation key, which the QE report binds; the QE report is signed by the platform's PCK, whose certificate chain in the quote must lead to Intel's SGX root CA, shipped with the library. A TD in debug mode is rejected.

In both cases the verifier checks the report data against the nonce and digest and compares the launch measurement with the measurements pinned in its configuration; a report that verifies without a pinned measurement produces a warning.

4.9 Monitoring Between Audits

An audit sees one moment. Code that ran and was removed before the audit leaves no file to find. The monitor runs a `bpftrace` program that records every program started and, where the kernel supports function tracing with BTF, every file mapped executable. The next audit includes a summary of that window: each distinct path, how often and by which users it was seen, and its current hash and owning package, or an error when the file is gone. The verifier explains these files like running code, and reports programs that ran and are no longer on disk. The log is written by root on the audited machine, so it is software evidence; IMA provides the same record with hardware backing.

4.10 Audit Status: Attester, Verifier and Transports

Audit Status packages Attestium as a single executable (Node.js single executable application) for Linux x64, Linux arm64 and macOS arm64, and as a container image, with published SHA-256 checksums and build provenance for every release. The verifier checks the attester's own executable, as reported in the evidence, against the published checksums of the release it claims to be.

The attester accepts exactly three operations: `check <nonce>` (collect evidence), `enroll` (return the AK and EK, creating the AK if needed), and `activate <credential>` (complete credential activation). It reads its configuration from a root-owned file and accepts nothing else from the verifier. Two transports carry these operations:

- **SSH:** the verifier's key is installed with `command="auditstatus ssh",restrict in authorized_keys`, so the key can run only the attester. Host keys must be pinned in a `known_hosts` file; an unknown or changed host key fails the connection.
- **Kubernetes:** the attester runs as a DaemonSet (one pod per node) and serves the same operations on the pod's loopback interface (`auditstatus serve`). The verifier reaches it with `kubectl port-forward`, using a service account that may only list the attester pods and port-forward to them. No Service or open port is needed.

A third mode, `local`, runs the attester in the verifier's own process when both are the same machine. On servers, the attester is granted `CAP_SYS_PTRACE` and `CAP_DAC_READ_SEARCH` so that it can read other users' processes and files without running as root. An Ansible role installs the attester, its account, its restricted keys, its capabilities and the optional monitor service; a Helm chart installs the DaemonSet with optional TPM and IMA access. `auditstatus init` inspects a repository and writes a starting verifier configuration, an attester configuration and a scheduled workflow; `auditstatus doctor` checks, on either side, that every check can run.

The verifier (`auditstatus verify`) runs the servers in parallel. For each it sends a fresh nonce, collects evidence, appraises it and writes the reports. Every finding has a severity: `fail` (the server differs from its references or the evidence is invalid), `warn`, `error` (the verifier could not complete a

check, which makes the result inconclusive) and `info`. With a retry delay configured, a server that fails or is inconclusive is collected again, because a deployment in progress looks like tampering; the second result is reported, and the first attempt’s findings remain in it as a warning. Values from evidence are escaped before they reach the Markdown report, so a server cannot inject links or markup into a published page.

4.11 The Core Library API

Beneath the evidence format, the main module keeps the lower-level primitives: file manifests with SHA-256 and git blob ids, Ed25519-signed baselines and comparisons with them, challenge-response with signed verification responses, a single-manifest TPM attestation with qualifying data `SHA-256(nonce || report digest)`, and runtime tracking that records the hash of each CommonJS module’s source as it is compiled. Configuration is data only: JavaScript configuration files are never loaded, because they would run code from the tree being verified.

5 Security Architecture

The security of the system rests on three things: where each fact comes from (the attester, the kernel, or the hardware), what each reference proves, and what the verifier refuses to accept. This section states each precisely, including what a result does not prove.

5.1 Cryptographic Primitives

- **SHA-256:** file, package, memory region, manifest and evidence digests; TPM qualifying data. Comparisons use SHA-256, with one exception: Composer packages are compared with the git tree of the commit `composer.lock` pins, by git object ids (SHA-1 in most repositories), because the commit names that tree and no other hash of the files is published. Git blob ids are otherwise recorded only to locate files in git trees.
- **SHA-512:** confidential VM report data; npm and NuGet integrity hashes as lockfiles record them.
- **Ed25519:** signed baselines and challenge responses over canonical JSON, and minisign checksum signatures.
- **TPM 2.0 quotes:** RSA (PKCS#1 v1.5) or ECDSA signatures by a TPM-resident Attestation Key over `TPMS_ATTEST`.
- **Confidential VM reports:** ECDSA P-384 (SEV-SNP) and ECDSA P-256 (TDX), with X.509 chains to AMD’s and Intel’s roots.
- **Sigstore and TUF:** X.509 chains to Sigstore’s certificate authority, transparency log signatures and inclusion proofs, DSSE envelopes, and threshold-signed TUF metadata.
- **OpenPGP:** signatures on distribution archive indexes, Node.js checksum lists and project checksum lists, verified with `gpgv` against configured keyrings.

All verification of TPM quotes, X.509 chains, confidential VM reports, Sigstore bundles and TUF metadata is implemented in JavaScript on top of Node.js's `crypto` module, so a verifier needs no vendor tooling.

5.2 Evidence Levels

Every check that runs on the audited machine depends on software that the machine's root user controls. That sets a hard limit on what software alone can prove. The verifier therefore assigns each result an evidence level, named after the hardware statements that verified: `software`, `tpm`, `tpm+ima`, `sev-snp` or `tdx`, or a combination such as `tpm+ima+sev-snp`.

5.2.1 Software Evidence

The attester's report, bound to the nonce by the digest and appraised against independent references.

- **Proves:** that whatever produced the evidence answered this nonce within the time window; and, if the attester reported honestly, that every compared file, package, image file and mapped library matched its reference.
- **Detects reliably:** drift, failed or partial deployments, unreviewed changes, stale processes, modified files and packages, injected libraries and preloads, attached debuggers and open inspectors, code that differs from the distribution package or official release it claims to be, and attacks by anyone without root on the machine.
- **Does not prove:** anything against an attacker with root who anticipates the audit. Such an attacker can run a modified attester, or feed the real one false data, and report whatever the verifier expects. The attester's own hash is checked against its published release, but a modified attester can report a false hash.

5.2.2 TPM-Bound Evidence

The same evidence, with a TPM quote over `SHA-256(nonce || evidenceDigest)` signed by an enrolled Attestation Key.

- **Proves:** that the evidence was produced after the nonce was chosen, on the machine whose TPM holds the enrolled key (and, with EK certificate enrollment, that this is a genuine TPM), and that the platform's PCR values were those quoted. When expected PCR values are pinned, it proves the machine booted the measured firmware, bootloader and kernel configuration those values describe.
- **Does not prove:** that the files, packages and processes reported are true. Everything after boot is still reported by software that root controls. A quote without pinned PCR values proves identity and freshness only.

5.2.3 TPM with IMA

The kernel measures files as they are executed, mapped executable or read, according to its policy, and extends each measurement into PCR 10 before the file is used. The verifier replays the log to the quoted PCR 10.

- **Proves:** that the measured files had the logged contents when the kernel loaded them, even against a hostile root user, because an entry cannot be removed from a PCR and the replay must match the signed value. The verifier compares the measurements of the service’s files with the public commit: a file whose last measurement differs fails, and earlier contents of the same path are reported even if the file was restored afterwards.
- **Does not prove:** anything about files outside the IMA policy; anything the kernel did not measure (code injected into memory without a file, JIT-compiled code); or anything against a compromised kernel or firmware, which the boot measurements are meant to reveal only when their expected values are pinned. The verifier compares the kernel’s measurements with the service’s own files and with the hashes reported for each process’s executables and libraries; other files, such as installed packages that no process maps, are still hashed by the attester.

5.2.4 Confidential Virtual Machine

A SEV-SNP or TDX report with report data `SHA-512(nonce || evidenceDigest)`, signed by a key that chains to the CPU vendor’s root.

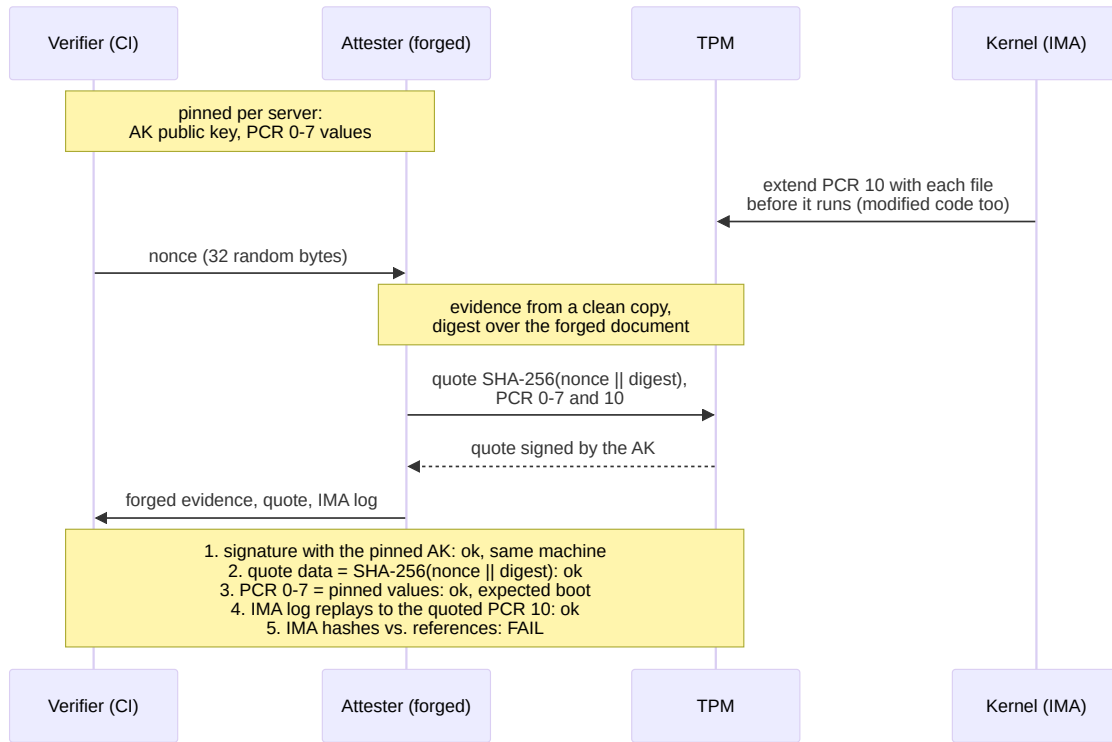
- **Proves:** that the evidence came from a guest running on genuine AMD or Intel hardware with confidential computing enabled and debugging disabled, after the nonce was chosen, and that the guest’s launch measurement is one the verifier pinned. The host operator, including a cloud provider, cannot read the guest’s memory or forge the report.
- **Does not prove:** anything beyond the launch measurement. The measurement covers the initial image (firmware and, depending on the setup, the kernel and initial RAM disk); what the guest loads later is reported by software inside the guest, whose root user controls it. For TDX, the platform’s TCB level is reported but not evaluated against Intel’s TCB information, so a platform with known but unpatched vulnerabilities is not rejected on that ground.

5.3 Forged Answers

The verifier asks the attester for evidence over SSH, as a user whose key may run only the attester, with the server’s host key pinned (or, on Kubernetes, through a port-forward to the pod’s loopback interface, authenticated by the cluster). There is no HTTP endpoint: it would answer anyone who can reach it, rely on any public certificate authority, and could be answered by a proxy in front of the server. But no transport makes the answer true. Whoever controls the server can replace the attester with a program that returns what the verifier expects: it copies the verifier’s nonce, hashes

a clean copy of the published commit instead of the deployed files, sets `collectedAt` to the current time, and computes `evidenceDigest` over the forged document. Every software check passes. The nonce proves only that the answer was made after the nonce was chosen; the digest is a hash, not a signature, so anyone can compute it; a software signing key on the server can be read by root; a TLS certificate or SSH host key authenticates the machine, not the software that answers. A result backed only by software evidence therefore states that it holds only while the attester is honest.

A forged answer fails only when part of it comes from something the forger cannot control. The diagram shows where a forgery fails when the server has a TPM with an enrolled key and IMA.



1. **The key cannot be copied.** The attestation key is created in the TPM as a restricted, non-exportable signing key, enrolled through the endorsement certificate and credential activation, and pinned per server. A quote from another machine, or a signature by a software key, fails.
2. **The quote covers this nonce and this document.** Root can make the real TPM quote a forged document; that ties the forgery to this machine and this moment, and leaves its content to the next steps.
3. **PCRs can be extended, never set.** A modified kernel or boot loader gives other PCR 0 to 7 values than the pinned ones.
4. **The IMA log cannot be edited.** An entry removed or changed makes the replay differ from the quoted PCR 10.
5. **Modified code leaves its hash.** The kernel measured the modified file before it ran. The

forged evidence reports the commit’s hash; the quoted log holds another, and the result fails. Files under the service’s root are compared with the commit or the package’s reference, never with the hashes the attester reports.

This holds only for what the IMA policy measures, and the verifier cannot see the policy. The common `tcb` policy measures programs executed, files mapped executable and files read by root; the scripts of a Node.js, Python or Ruby service running as another user are read, not executed, and are never measured. The policy must also measure the service user’s reads (`measure func=FILE_CHECK mask=~MAY_READ uid=<uid>`), by the reading user rather than the file’s owner, since root can change an owner. The verifier reports a service under whose root nothing was measured, and fails it when IMA is required; the policy is bound to the boot measurements by loading it from the initramfs and pinning the PCRs that measure the initramfs and kernel command line.

Relaying the nonce to an honest machine that runs the published code passes with software evidence, because nothing in it names the machine. It fails with a TPM quote when each server has its own pinned key. With a confidential VM, it passes when the honest guest was started from the same image, since the launch measurement identifies an image, not an instance.

What remains possible against TPM and IMA is listed under each level above: code the IMA policy does not measure, code run from outside the service’s root by a process the evidence leaves out (the IMA log does not say who read a file), facts no measurement covers (such as which processes run), and a compromised kernel or firmware. The verifier decides the required level for each server from its own configuration, never from the evidence, so a server that omits its quote fails instead of falling back to software evidence.

5.4 What Each Reference Proves

Matching a reference proves that a file is the same as that reference, and nothing more. The strength of the conclusion depends on what the reference itself shows.

- **Public commit:** the deployed file is the file at a commit that is on the public branch. It does not prove the commit was reviewed or benign.
- **Reproduced build:** the generated file is what the build of that commit produces on the verifier. It relies on the build being reproducible [25], [27].
- **Release manifest with attestation:** the files are those a named workflow built from a named commit. It relies on that workflow and its build environment.
- **Lockfile-pinned package:** the installed files are those of the artifact whose hash the lockfile at the commit pins. It does not prove that the artifact was built from its project’s public source; npm provenance, where a package publishes it, links the tarball to a repository and commit. A jar that the build does not pin is compared with Maven Central by coordinates, which trusts the registry instead of the repository.

- **Container image by digest:** the container’s files are the image’s files. With an attestation, the image was built by a named workflow; without one, the image is only what the registry holds under that digest.
- **Official release, signed checksum list, distribution package:** the binary is the one its publisher signed. It is as trustworthy as that publisher’s signing key.
- **Go and Rust build information:** the dependencies the build recorded are the ones the lockfile pins. Because the build writes this record, it shows nothing about the binary unless the binary’s own hash is matched by one of the references above.

Some files have no possible reference, and are reported as such instead of passing: Python byte-code caches, native extensions compiled on the server, packages built from source distributions, dependencies from unpinned sources, and generated files such as Composer’s autoloader unless a reproduced build covers them.

5.5 What the Verifier Refuses

- Evidence that fails the schema, carries another nonce, is outside the time window, or whose digest does not match.
- Hardware statements verified with keys taken from the evidence. The verifier uses only attestation keys it pinned at enrollment and vendor roots it ships.
- References that are not authenticated by something it already trusts: archive indexes whose signature fails, registry artifacts whose hash differs from the lockfile, image layers whose digest differs, bundles whose certificate identity differs.
- Incomplete checks as passing. A process with any check in `incomplete`, a reference that could not be fetched, or a missing hardware statement from a machine configured to require it makes the result fail or inconclusive.
- Code from the audited server. The verifier never loads configuration or code from the evidence; a reproduced build runs only for commits on the audited branch of the public repository, with a minimal environment.

5.6 Process Memory Integrity

File-level verification answers whether the file on disk is correct. It cannot answer whether the code executing in memory is the code on disk. An attacker with root, or with `ptrace` access to a process, can modify its executable memory without touching any file [17]:

- **`ptrace` injection:** attaching to a process and writing code into its address space.
- **`/proc/<pid>/mem` writes:** writing to a process’s memory through the `proc` filesystem.
- **Linker hijacking:** `LD_PRELOAD`, `LD_AUDIT` or `/etc/ld.so.preload`, which load a library before all others.
- **Fileless payloads:** `memfd_create` objects that are executed without ever existing on disk.

- **Runtime-specific paths:** code loaded through a runtime’s own options or environment (`NODE_OPTIONS`, `PYTHONPATH`, `JAVA_TOOL_OPTIONS` agents, `RUBYOPT`, `.NET` start-up hooks), or a debugger or inspector opened at runtime and used to evaluate code.

The process integrity checks detect each of these, and the test suite demonstrates each against a live process: a byte of `libc` modified through `/proc/<pid>/mem`, preloads, an inspector opened with `SIGUSR1`, a `ptrace`-attached tracer, and a `memfd` payload. Code produced by a JIT compiler lives in anonymous memory with no file to compare against. For interpreted and JIT-compiled languages, what matters is the source that was loaded; that is covered by comparing the source with the public commit, by reproducing generated files, by reporting files changed after the process started, and, with IMA, by the kernel’s own measurements.

5.7 Threat Model

The system considers the following adversaries.

1. **External attacker without root,** for example through a remote code execution vulnerability in the application. The ways such an attacker changes what runs are reported: modified files and packages, preloads and runtime injection options, debuggers and inspectors, `memfd` payloads, and processes whose files changed after they started. Code kept only in a runtime’s heap is not visible to memory comparison. A process started outside the service’s directory under another user is not inspected, although the monitor records its execution.
2. **Attacker or insider with root.** Software evidence alone does not hold: root can replace the attester or the data it reads. Mistakes and casual changes are still reported. A deliberate attacker with root is detected only by hardware-backed evidence: IMA measurements of the files the kernel loaded, bound by a TPM quote, which root cannot erase. An attacker who can load a malicious kernel module or boot a modified kernel is detected only if the boot measurements are pinned and the kernel’s integrity controls prevent unmeasured changes.
3. **Hosting or cloud operator.** An operator with physical or hypervisor access can read and modify a normal VM’s memory. A confidential VM prevents this, and its report proves the launch measurement; the guest’s later state is still reported by software inside it.
4. **Supply chain attacker,** who alters a package in a registry, a mirror or on disk. Lockfile-pinned hashes and file-by-file comparison detect any change after the lockfile was written. A malicious version that was published, reviewed and pinned before the change is outside this check; provenance attestations narrow the gap for packages that publish them.
5. **Compromised reference source.** The verifier trusts the public repository host, the registries through the lockfile’s hashes, the distributions’ signing keys, the Sigstore trust root, the TPM manufacturers’ and CPU vendors’ roots, and its own CI environment. A compromise of

any of these affects results that depend on it. Content-addressed references (lockfile hashes, image digests, git commits) limit what a compromised transport can do.

6. **Compromised verifier.** A verifier that lies can publish false results. The verifier’s code, configuration, logs and results are public, and anyone can run the same verifier against the same server if given access, but the results themselves are not yet signed or recorded in a transparency log (Section 8).
7. **Changes between audits.** A change made and undone between two audits leaves no file to find. The monitor records what ran between audits, with the strength of software evidence; IMA records the same with hardware backing until the next reboot.

Each layer narrows what an attacker can do unnoticed. Only hardware-backed evidence holds against an attacker who already controls the machine, and every result states which level it reached.

6 Our Security Background

Attestium grew out of the operation of Forward Email, an open-source email service whose production environment is managed entirely through open-source Ansible playbooks that anyone can inspect. This section describes that hardening, and why it is paired with point-in-time audits and continuous automated verification.

6.1 Ansible-Managed Infrastructure

Every server we operate is provisioned and hardened through Ansible. Security measures are codified, version-controlled, and reproducible rather than applied by hand. Here’s what our playbooks enforce.

6.1.1 Kernel and System Hardening

We disable core dumps entirely — hard and soft limits set to zero, `fs.suid_dumpable=0`, `Process-SizeMax=0` in `systemd`, and `kernel.core_pattern` piped to `/bin/false`. Transparent Huge Pages are disabled via a `systemd` service. Swap is turned off on most servers; our MongoDB, Valkey and logs servers keep a small emergency swap with `vm.swappiness=1`. Every server gets more than 40 `sysctl` kernel parameters, and database servers get more, including ASLR (`kernel.randomize_va_space=2`), TCP SYN cookies for flood protection, RFC 1337 TIME-WAIT assassination protection, and TCP buffers scaled to available RAM.

`/dev/shm` is mounted as `tmpfs` with `noexec,nosuid,nodev`, so code cannot execute from shared memory. The root filesystem is mounted with `noatime`, `nodiratime` and `discard`. On database data drives we set the I/O scheduler by device type (`none` for multi-queue devices such as NVMe,

deadline otherwise) and tune read-ahead. Web, API, CalDAV and CardDAV servers and database servers use TCP BBR congestion control; mail servers use CUBIC with `fq_codel`.

6.1.2 USB Device Whitelisting

The `usb-storage` kernel module is disabled via `modprobe.d` and the `initramfs` is rebuilt to persist this across reboots. We maintain a whitelist of authorized USB devices by `vendor:product` ID in `/etc/security-monitor/authorized-usb-devices.conf`. Any unrecognized USB device triggers an immediate email alert to the team. Udev rules (`99-usb-monitor.rules`) provide real-time detection on top of the 5-minute polling cycle. A datacenter technician plugging in a USB drive gets flagged instantly.

6.1.3 SSH and Access Control

Root login is disabled (`PermitRootLogin no`). Password authentication is disabled — only key-based auth is allowed. The root password is locked. We maintain two users: `devops` (with `sudo`, no password prompt) and `deploy` (with a 4096-bit SSH key, limited privileges). Fail2Ban is configured aggressively: 2 failed attempts trigger a permanent ban (`bantime=-1`) with a 365-day find window, and bans are shared across all of our servers. Alerts go out through a send-only Postfix configuration that listens on no TCP port, accepts mail only from root, and delivers straight to the recipient's mail server with no relay credentials.

6.1.4 Nine Security Monitoring Systems

We run nine independent monitoring systems, all implemented as `systemd` timers with rate-limited email alerts:

1. **System Resource Monitor** — CPU, memory, and disk usage with five threshold levels (75%, 80%, 90%, 95%, 100%), checked every 5 minutes.
2. **SSH Security Monitor** — Failed logins, successful logins, root access, unknown IPs, and after-hours logins, checked every 10 minutes.
3. **USB Device Monitor** — Unknown device detection with `vendor:product` ID whitelisting, checked every 5 minutes plus real-time udev rules.
4. **Root Access Monitor** — Direct root login, `sudo` usage, `su` to root, and privilege escalation attempts, checked every 5 minutes.
5. **Lynis Audit Monitor** — Automated Lynis security audits, run daily.
6. **Package Monitor** — Tracks every package installation and removal, checked hourly.
7. **Open Ports Monitor** — Detects unexpected listening services, checked hourly.
8. **SSL Certificate Monitor** — Checks TLS certificate expiry dates daily.
9. **Outbound Traffic Monitor** — Alerts on sustained outbound traffic above a set limit, checked every minute.

Each monitor uses whitelist files in `/etc/security-monitor/` for authorized IPs, users, USB devices, root users, and sudo users. Rate limiting prevents alert flooding — resource alerts have a 1-hour cooldown per threshold, SSH root access alerts have no cooldown (always alert), and USB alerts have a 1-hour cooldown per device.

6.1.5 Command Logging and Auditing

Every command executed on our servers is logged through multiple layers: `auditd` with custom audit rules, enhanced bash logging in both `/etc/profile.d/` (login shells) and `/etc/bash.bashrc`, zsh logging in `/etc/zsh/zshrc.d/`, and `rsyslog` capturing everything to `/var/log/bash-commands.log` with 30-day logrotate retention. If someone runs a command on our servers, we have a record of it.

6.1.6 Automatic Security Updates

Unattended upgrades install security patches automatically. Servers reboot at 08:00 UTC when an update requires it, never while someone is logged in; our MongoDB, Valkey and logs servers do not reboot automatically. We deploy a custom port scan protection script (our own maintained fork). DNS resolves through a local Unbound caching resolver that forwards to Cloudflare and Google over DNS-over-TLS. MongoDB is installed from the official repository and Valkey is compiled from source. Our only external Ansible code is three roles (DNS, fail2ban and sysctl) and three collections, each pinned to an exact version in `requirements.yml`.

6.2 Audits and Continuous Verification

A one-time security audit from a reputable firm costs \$5,000–\$10,000 USD or more. That buys you a snapshot — a report that says “on this date, we checked these things and they looked fine.” The moment the audit ends, the report starts going stale. New code gets deployed, packages get updated, configurations change. The audit doesn’t tell you what happened last Tuesday at 3 AM.

But the cost isn’t the real problem. The real problem is trust.

We manage email. Email is the most sensitive communication channel most people have — password resets, financial statements, legal correspondence, medical records. Giving a third-party auditor SSH access to our production servers means trusting them with access to all of that. Even with the best intentions, an audit can take weeks or months. During that entire window, we’d need to monitor their access, verify they aren’t exfiltrating data, and hope that their own systems haven’t been compromised. We’d essentially need to audit the auditor.

We have since completed a third-party security audit by Cure53 and published the report. Audits and continuous verification answer different questions. An audit is a deep, expert review of the code and systems at one point in time. Continuous verification checks the running state after every deploy and between deploys. It runs on a schedule and needs no interactive access to the servers:

the verifier's SSH key can run only the attester, which accepts three operations (collect evidence for a nonce, and the two steps of TPM enrollment) and nothing else.

6.3 From Hardening to Verification

The hardening described above (the `sysctl` parameters, USB whitelisting, the nine monitoring systems, command logging) consists of preventive controls. They make an attack harder, but they do not answer the question a user of the service cares about: is the code running on this server now the code in the public repository?

Attestium answers that question. The Ansible playbooks are designed to prevent unauthorized changes; Attestium and Audit Status are designed to detect them, on a schedule, with results that anyone can check against the public repository and the public releases the service depends on. Process memory integrity extends the question to the code executing in memory, and hardware-backed evidence extends it to an attacker who has root on the server. On a hardened server, the attacker to plan for is not only one who modifies files, but one who modifies memory or the tools that report on it.

7 The Attestium Stack in Context

Attestium builds on existing work in system transparency, remote attestation and supply chain security. This section describes how it relates to that work, and then the components it is built from.

7.1 Related Work

Project	Focus	Relationship to Attestium
System Transparency	Verifiable boot of a known operating system image	Attestium extends the same goal to the running application: its files, its executable memory, its dependencies and the libraries it maps.
Keylime	TPM and IMA remote attestation, with measured boot and runtime policies	Keylime attests the platform against IMA allowlists and boot policies. Attestium derives the expected value of each running file from public references (commits, lockfiles, images, releases, distribution archives) and adds process memory comparison, runtime injection checks and confidential VM reports.
Sigstore	Signing and verifying software artifacts	Attestium verifies Sigstore bundles as references, and checks what is installed and running after deployment.
SLSA and in-toto	Provenance and integrity of the build	They describe how an artifact was produced; Attestium checks that the deployed result still matches the artifact and that the running processes match the deployed files.
Reproducible Builds	Independently rebuilding artifacts from source	Attestium uses reproduced builds as references for generated files, and depends on reproducibility for any build output without an attestation.
Socket.dev	Analysis of what packages do	Socket judges package behavior; Attestium confirms that the installed files are exactly the artifacts the lockfile pinned.

System Transparency [6], [7], [28] provides verifiable boot for Mullvad’s VPN servers: a signed, reproducible operating system image, booted without persistent state. Attestium takes the same goal, that users can verify what a server runs, and applies it to the application layer, where most services change several times a day.

Keylime [29] is a CNCF project for TPM-based remote attestation, including measured boot and IMA runtime policies. Its policies are lists of expected file hashes that an operator maintains. Attestium obtains the expected value of every running file from a public reference instead, which is what lets a third party with no relationship to the operator perform the check.

Sigstore [4], [15], **SLSA** [3] and **in-toto** [14] secure the path from source to artifact. Attestium consumes their output: GitHub artifact attestations and npm provenance are Sigstore bundles carrying in-toto statements with SLSA provenance, and Attestium verifies them as references for release manifests, container images and packages.

Reproducible Builds [25] ensure that a given source always produces the same binary, which lets independent parties confirm that a binary came from its claimed source [27]. Attestium’s

comparisons are the runtime counterpart: they confirm that the binaries and packages on a server are the published ones, and that the processes in memory have not been modified since they were loaded.

The **IETF RATS** architecture [10] provides the vocabulary: attester, verifier, relying party, evidence, endorsements, reference values and attestation results. The **SCITT** architecture [30] describes transparency services for signed supply chain statements, a natural place to record attestation results (Section 8).

7.2 Architecture Overview



7.3 Implementation

Attestium is a Node.js package (Node.js 18 or later) with two runtime dependencies: **cosmiconfig** for data-only configuration files and **js-yaml**. Everything a verifier needs to authenticate a reference is implemented in the library in JavaScript on top of Node.js's **crypto** module: parsing of TPM structures, X.509 path validation for TPM manufacturer, AMD and Intel chains, SEV-SNP and TDX report verification, Sigstore bundle verification, a TUF client, minisign, ASN.1, ELF, tar, zip and TOML readers, and a JSON Schema validator for the subset the evidence format uses. Every parser that reads data from the audited machine is bounds-checked. Downloads go through a hardened HTTP client: HTTPS only (plain HTTP only for loopback hosts, or for content authenticated by signature), bounded redirects that never forward credentials to another host, size limits, timeouts, and retries with back-off.

External programs are used only where a system interface requires them, always without a shell and with bounded time and output: **tpm2-tools** on the attester for TPM operations; **gpgv** on the verifier for OpenPGP signatures; **git** on the verifier for blobless clones of public repositories, with hooks, prompts and system configuration disabled; the Docker Engine API or **crictl** on the attester for container metadata; and **bpftool** for the optional monitor. Nothing the attester collects is executed or loaded; binaries are hashed and scanned for build information, never run.

The test suite requires full statement, branch, function and line coverage. The tests are end-to-end: they attack real processes, serve real release archives, registry artifacts and container images from local servers, and use a software TPM.

Audit Status is built on Attestium and **js-yaml**. It is distributed as a Node.js single executable application for Linux x64, Linux arm64 and macOS arm64, and as a container image, with **SHA256SUMS** and GitHub build provenance for each release. Deployment tooling consists of an Ansible role for

servers, a Helm chart for Kubernetes clusters, a systemd unit for the monitor, and a GitHub Action for the verifier. Configuration for both roles is YAML with strict validation: unknown keys are errors, so a misspelled setting cannot silently disable a check.

8 Roadmap

This section lists what is implemented and what remains. The remaining items follow from the limits stated in Section 5 and from what the code reports as unsupported or unverifiable.

8.1 Implemented

- **Open evidence format:** Evidence version 2, with a specification, a JSON Schema, a canonical digest, and binding of the nonce and digest into TPM quotes and confidential VM reports.
- **Attester and verifier separation:** Collection and appraisal are separate functions for every check, so evidence collected on one machine is appraised on another against references the verifier obtains itself.
- **Process memory integrity:** Byte-for-byte comparison of every file-backed executable mapping with its file, including processes in containers; detection of linker injection, `memfd` payloads, tracers, and debuggers or inspectors opened at runtime.
- **Runtime profiles:** Code-injection vectors and debug ports of Node.js, Python, the JVM, Ruby, .NET, Erlang/Elixir, PHP, Perl, Deno and Bun.
- **Explanation of every executable and library:** Service files, packages, image files, official Node.js releases, pinned hashes, signed checksum lists, and Debian and Ubuntu packages from the signed archive, with snapshot lookups for superseded versions; unexplained files are reported.
- **Package ecosystems:** npm, PyPI, RubyGems, Hex, Composer, Maven and NuGet installations compared with the artifacts their lockfiles pin; Go and Rust dependencies compared inside the binary.
- **Build output:** Reproduced builds of the deployed commit for files the repository ignores.
- **Releases without git:** Release manifests attested with GitHub artifact attestations.
- **Sigstore and provenance:** Bundle verification with a TUF client for Sigstore’s trust root; npm provenance; container image attestations as OCI referrers.
- **Containers:** Image, mounts, writable layer and root filesystem compared with the image fetched from its registry by digest.
- **TPM 2.0:** Attestation keys, quotes verified in pure JavaScript, and enrollment through endorsement key certificates and credential activation.
- **IMA:** Log replay to the quoted PCR 10, and comparison of the kernel’s measurements of service files with the public commit.
- **Confidential VMs:** AMD SEV-SNP and Intel TDX reports through `configs-tsm`, verified

to the vendors' roots, with pinned launch measurements.

- **Monitoring between audits:** An eBPF record of programs executed and files mapped executable since the previous audit.
- **Transports and deployment:** SSH forced commands with pinned host keys; a Kubernetes DaemonSet reached through port-forward; an Ansible role, a Helm chart, a GitHub Action, and setup and diagnostic commands.

8.2 Remaining Work

8.2.1 Hardware and Platform

- **Measured boot policies:** Expected PCR values are currently pinned by hand. Parsing the TCG UEFI event log and appraising each boot event (firmware, bootloader, kernel, command line, Secure Boot state) against reference values would make boot measurements maintainable across kernel and firmware updates.
- **IMA for all code:** Use the kernel's measurements, not only the attester's report, to explain every executable and library, and verify IMA file signatures (`ima-sig`) where distributions sign their files. With that, the monitor's record between audits would also have hardware backing.
- **Confidential computing:** Evaluate the TDX platform TCB level against Intel's published TCB information (currently reported, not evaluated), support minimum TCB policies for SEV-SNP, derive expected launch measurements from published firmware and images, and support further environments such as Arm CCA and AWS Nitro Enclaves.
- **Other operating systems:** Evidence collection is complete on Linux. macOS and Windows support the subset of process checks their tooling exposes.

8.2.2 References

- **More distributions:** Only dpkg-based distributions (Debian and Ubuntu) are explained from their archives. RPM-based distributions (Fedora, Red Hat Enterprise Linux and its rebuilds, openSUSE) with signed repository metadata, Alpine's `apk`, and Arch Linux remain.
- **More registries and lockfiles:** Yarn and Bun lockfiles for JavaScript; CPAN for Perl, whose runtime is profiled but has no package plugin; Conda; Swift and Dart packages; and git dependencies hosted outside GitHub, which are currently unverifiable.
- **Code built on the server:** Native extensions compiled at install time, packages built from source distributions, and bytecode caches have no reference. Reproducing them in the verifier, as is done for configured build outputs, would remove these exceptions.
- **Wider provenance:** Verify the provenance attestations that registries other than npm publish, such as PyPI's, and require provenance by policy for selected dependencies.
- **Certificate transparency:** Sigstore signing certificates are verified through the trans-

parency log entry, but their embedded certificate transparency timestamps are not.

8.2.3 Results

- **Signed and logged attestation results:** Verifier reports are published as files in a git repository. Signing each result and recording it in a transparency log, such as a SCITT transparency service [30], would let relying parties check that the history of results has not been rewritten, and would let several independent verifiers publish results for the same server.
- **Standard result formats:** Express results in the attestation result formats defined by the IETF RATS working group, so other relying parties can consume them.
- **Attested verifiers:** Run the verifier in a confidential VM, so that its own execution can be attested to relying parties.

8.2.4 Limits of the Approach

Some limits are not expected to go away. Code produced by a JIT compiler has no file to compare with. Software evidence cannot be made to hold against root on the audited machine; only hardware-backed evidence can. And a reference only proves sameness: a service that faithfully runs malicious published code passes. Attestium shows that what runs is what was published; reviewing what was published remains necessary.

8.3 Community and Collaboration

Attestium and Audit Status are open-source projects. Contributions of ecosystem plugins, distribution references and attestors or verifiers in other languages are welcome; the evidence format is designed so that independent implementations interoperate.

9 Adoption

Attestium is intended for any service whose users should be able to check that it runs its published open-source code. This section describes the reference deployment at Forward Email, how other services adopt the same approach whatever their language and platform, and the insider threat that motivates it.

9.1 Reference Adopter: Forward Email

Forward Email is an open-source email service. Its production servers (web, API, IMAP, POP3, SMTP, MX, CalDAV, CardDAV, job processing and SQLite storage) run Node.js applications from its public repository. Each server is deployed from git: a deploy checks out the commit, installs dependencies with the npm version the repository pins and a frozen lockfile, builds the browser assets and the Sieve parser, and reloads the PM2 processes. The results are published at:

<https://status.forwardemail.net>

The verification is the first project of the Audit Status public registry, which verifies registered services as a third party:

- The **attester** is installed on each server by an Ansible playbook, as a dedicated account whose authorized keys are the registry's keys, restricted to the attester command. The same playbook records the servers' host keys, which the registry file pins, and can enroll each server's TPM.
- The **verifier** runs hourly in Audit Status's own GitHub Actions workflow. Its SSH key was generated on a GitHub-hosted runner and exists only as a secret of that workflow's environment. It compares each server with the latest GitHub release of the public `forwardemail.net` repository, the release's lockfile and the npm tarballs it pins, with their build provenance, the official Node.js release, and the global npm, pnpm and PM2 packages.
- Files the deployment generates and the repository ignores (the browser build and the generated Sieve parser) are compared with a build of the same commit that the registry reproduces in a separate job, without the SSH key, as an unprivileged account. The job that holds the key runs no code of the project.
- PM2's `node_args` and every process's runtime injection vectors are checked. A server whose deployment is in progress is collected again after ten minutes, and within an hour of a new release a server that still runs the previous one is a warning.
- The workflow publishes a JSON and a Markdown report for every run to the registry's `status` branch, signed with a GitHub artifact attestation, so the branch's history is a public record of the production state, and the badge on the status page reflects the latest result. When a server fails or the result is inconclusive, the workflow opens an issue in the Audit Status repository.

A result for a server without an enrolled TPM is at the **software** level, and the published report says so. Anyone can read the same reports the Forward Email team reads.

9.2 How Other Services Adopt

Adoption follows the same steps for any service:

1. **Describe the service.** `auditstatus init` inspects the repository, detects its languages, package managers and container build, and writes a verifier configuration, an attester configuration and a scheduled workflow.
2. **Install the attester.** On servers, with the Ansible role or the release binary and a restricted SSH key; on Kubernetes, with the Helm chart, which runs the attester as a DaemonSet and creates a service account that can only port-forward to it.
3. **Enroll hardware.** Run TPM enrollment for each server, or pin the expected launch mea-

surements of confidential VMs. This step is optional, but it determines the level of evidence a result can reach.

4. **Check the setup.** `auditstatus doctor` confirms, on each side, that every check can run: permissions, processes, ecosystems, containers, TPM, IMA, confidential VM, monitor, and access to every reference.
5. **Publish.** Add the service to the Audit Status public registry, which verifies it every hour from Audit Status’s own workflow, or run the verifier on a schedule in a public repository of the service. Link the badge from the service’s status page.

How each part of a service is verified depends on how it is built and deployed, not on its language:

- **Interpreted services deployed from git** (JavaScript, Python, Ruby, PHP, Elixir): the source is compared with the commit, installed packages with the lockfile’s artifacts, generated files with a reproduced build, and the interpreter with its official release or distribution package.
- **Compiled services** (Go, Rust, Java, .NET): the binary is compared with an attested release manifest, a signed checksum list or a reproduced build; the dependencies compiled into Go and Rust binaries are compared with `go.sum` and `Cargo.lock`; jars and .NET assemblies from registries with the hashes their builds pin.
- **Containers:** every file of each container is compared with its image, fetched by digest, optionally required to be attested by the project’s own workflow.
- **Third-party software:** databases, proxies and other programs installed from the distribution are explained by the distribution’s signed archive; others by a signed checksum list or a pinned hash.

Consider a hosted database platform similar to Supabase, whose open-source stack combines a PostgreSQL database with services written in TypeScript, Go and Elixir, often run as containers. Its PostgreSQL binaries and extensions are explained by their distribution packages or by the image they ship in; its container images by digest and by the attestations of the workflows that built them; its Go services by the build information checked against `go.sum` and by an attested binary hash; its Elixir services by the Hex packages `mix.lock` pins and a reproduced release build; and every process’s runtime injection vectors are checked whatever its language. With confidential VMs, the platform can additionally show that its hosting provider cannot read or modify customer data in memory.

9.3 A Call to the Open-Source Community

Many companies build their business on open-source services and ask users to trust that the hosted version is the published one. The following projects publish their source and would be natural candidates; for each, the website, primary repository and main languages are listed:

- **Supabase** (supabase.com) - github.com/supabase/supabase (TypeScript, Go)
- **Cal.com** (cal.com) - github.com/calcom/cal.com (TypeScript)
- **Documenso** (documenso.com) - github.com/documenso/documenso (TypeScript)
- **Bitwarden** (bitwarden.com) - github.com/bitwarden/clients (TypeScript)
- **Infisical** (infisical.com) - github.com/Infisical/infisical (TypeScript)
- **Jitsi** (jitsi.org) - github.com/jitsi/jitsi-meet (TypeScript, JavaScript)
- **Element** (element.io) - github.com/element-hq/element-web (TypeScript, CSS)
- **Mattermost** (mattermost.com) - github.com/mattermost/mattermost (Go, TypeScript)
- **Ghost** (ghost.org) - github.com/TryGhost/Ghost (JavaScript, TypeScript)
- **Plausible Analytics** (plausible.io) - github.com/plausible/analytics (Elixir, React)
- **PostHog** (posthog.com) - github.com/PostHog/posthog (Python, TypeScript)
- **Chatwoot** (chatwoot.com) - github.com/chatwoot/chatwoot (Ruby, Vue, JavaScript)
- **Twenty** (twenty.com) - github.com/twentyhq/twenty (TypeScript)
- **Gitea** (about.gitea.com) - github.com/go-gitea/gitea (Go, TypeScript)
- **Rocket.Chat** (rocket.chat) - github.com/RocketChat/Rocket.Chat (TypeScript)
- **Plane** (plane.so) - github.com/makeplane/plane (TypeScript, Python)

These projects span JavaScript and TypeScript, Go, Python, Ruby and Elixir, each of which Attestium covers. By publishing continuous, independently checkable results, such a service can show its users that the code on its servers matches its public repository and the releases it depends on, and, with a TPM and IMA or a confidential VM, that this holds even against someone with root access to the server or the host.

9.4 Preventing the Insider Threat

External attacks receive the most attention, but insiders are among the hardest threats to mitigate. An employee acting in bad faith, a compromised vendor, or a datacenter technician with physical access to a server can bypass many controls and change what a service runs.

With shell access, such a person can modify application files, install backdoors or alter dependencies. With more skill, they can work entirely in memory: `LD_PRELOAD` to hijack library calls, `ptrace` to inject code into running processes, a runtime's own loading options, or `memfd_create` to execute payloads that never touch the filesystem. File integrity monitoring does not see these.

Attestium targets this class of changes. It compares the deployed files with the public commit and the installed packages with the artifacts the lockfile pins; it compares the executable pages of every mapped file with the file on disk; it explains every running executable and library by a public reference; and it reports preloads, runtime injection options, debuggers, inspectors and `memfd` payloads.

This raises the bar for everyone with access, including the operator. Careless or casual changes

appear in the next public report. A determined insider with root can defeat software-only evidence, which is why the strongest results come from hardware: with IMA, the kernel records what was loaded before the insider can intervene, and the TPM will not sign a history that was edited afterwards; with a confidential VM, the host’s operators cannot read or change the guest’s memory at all.

10 Conclusion

Build-time security establishes what should run. Nothing in a typical deployment shows users what does run. Attestium closes that gap with an open evidence format and a verifier that compares every fact about a server with a reference the server’s operator does not control: the public commit, a reproduced build, the artifacts a lockfile pins, a container image by digest, an official release, a signed checksum list, or the distribution’s signed archive.

The approach does not depend on a language or a packaging method. Every executable and library that a process runs or maps is either explained by such a reference or reported as unexplained; every process is checked for the code-injection vectors of its runtime and for executable memory that differs from its files; and every check that cannot complete makes the result inconclusive, never passing.

The paper has also been precise about limits. Software evidence detects drift, failed deployments and tampering by anyone without root, but it cannot stand alone against an attacker who controls the machine. TPM quotes bind evidence to an enrolled machine and its boot state, IMA adds kernel measurements that root cannot rewrite, and confidential VMs remove the host operator from the trusted base. Every result states which of these it rests on.

Attestium is the building block; Audit Status is a ready-made attester and verifier on top of it; Forward Email’s public status page is a working deployment. The remaining work, including measured boot policies, more distributions and registries, and signed, logged attestation results, is listed in Section 8. Contributions, independent implementations of the evidence format, and additional adopters are welcome.

References

- [1] CISA, “Advisory AA20-352A: Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations.” 2020. Available: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a>
- [2] A. Freund, “backdoor in upstream xz/liblzma leading to ssh server compromise.” 2024. Available: <https://www.openwall.com/lists/oss-security/2024/03/29/4>
- [3] SLSA, “SLSA: Supply-chain Levels for Software Artifacts.” Available: <https://slsa.dev/>

- [4] The Linux Foundation, “Sigstore: A new standard for signing, verifying, and protecting software.” Available: <https://www.sigstore.dev/>
- [5] Sonatype, “9th Annual State of the Software Supply Chain.” 2023. Available: <https://www.sonatype.com/state-of-the-software-supply-chain/2023/introduction>
- [6] Mullvad VPN, “System Transparency is the future.” 2019. Available: <https://mullvad.net/en/blog/system-transparency-future>
- [7] Mullvad VPN, “Diskless infrastructure in beta (System Transparency: stboot).” 2022. Available: <https://mullvad.net/en/blog/diskless-infrastructure-beta-system-transparency-stboot>
- [8] Federal Register, “Executive Order 14028: Improving the Nation’s Cybersecurity.” 2021. Available: <https://www.federalregister.gov/documents/2021/05/17/2021-10460/improving-the-nations-cybersecurity>
- [9] NIST, “NIST SP 800-218: Secure Software Development Framework (SSDF) Version 1.1.” 2022. Available: <https://csrc.nist.gov/pubs/sp/800/218/final>
- [10] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, “Remote ATtestation procedureS (RATS) Architecture.” 2023. Available: <https://datatracker.ietf.org/doc/rfc9334/>
- [11] Trusted Computing Group, “Trusted Platform Module Library Specification, Family 2.0.” Available: <https://trustedcomputinggroup.org/resource/tpm-library-specification/>
- [12] Advanced Micro Devices, “AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More.” 2020. Available: <https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf>
- [13] Intel Corporation, “Intel Trust Domain Extensions (Intel TDX).” Available: <https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/overview.html>
- [14] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappel, “in-toto: Providing farm-to-table guarantees for bits and bytes,” in *Proceedings of the 28th USENIX Security Symposium*, 2019, pp. 1393–1410. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>
- [15] Z. Newman, J. S. Meyers, and S. Torres-Arias, “Sigstore: Software Signing for Everybody,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 2353–2367. doi: 10.1145/3548606.3560596.
- [16] Google Cloud, “Binary Authorization for Borg.” Available: <https://cloud.google.com/docs/security/binary-authorization-for-borg>
- [17] Red Canary, “Playing with Process Memory Integrity on Linux.” Available: <https://redcanary.com/blog/threat-detection/process-memory-integrity-linux/>

- [18] GitHub, “Introducing npm package provenance.” 2023. Available: <https://github.blog/security/supply-chain-security/introducing-npm-package-provenance/>
- [19] GitHub, “Using artifact attestations to establish provenance for builds.” Available: <https://docs.github.com/en/actions/security-for-github-actions/using-artifact-attestations>
- [20] The Update Framework, “The Update Framework Specification.” Available: <https://theupdateframework.github.io/specification/latest/>
- [21] Debian Wiki, “SecureApt.” Available: <https://wiki.debian.org/SecureApt>
- [22] Open Container Initiative, “OCI Image Format Specification.” Available: <https://github.com/opencontainers/image-spec>
- [23] R. Sailer, X. Zhang, T. Jaeger, and L. van Doorn, “Design and Implementation of a TCG-based Integrity Measurement Architecture,” in *Proceedings of the 13th USENIX Security Symposium*, 2004, pp. 223–238. Available: <https://www.usenix.org/conference/13th-usenix-security-symposium/design-and-implementation-tcg-based-integrity-measurement>
- [24] Linux Kernel Documentation, “configs-tsm: Trusted Security Module report interface.” Available: <https://www.kernel.org/doc/Documentation/ABI/testing/configs-tsm>
- [25] Reproducible Builds, “Reproducible Builds: Increasing the integrity of software supply chains.” Available: <https://reproducible-builds.org/>
- [26] Linux Kernel Documentation, “IMA Template Management Mechanism.” Available: <https://www.kernel.org/doc/html/latest/security/IMA-templates.html>
- [27] C. Lamb and S. Zacchiroli, “Reproducible Builds: Increasing the Integrity of Software Supply Chains,” *IEEE Software*, vol. 39, no. 2, pp. 62–70, 2022, doi: 10.1109/MS.2021.3073045.
- [28] R. Dahlberg, “Hold on to your hat and learn System Transparency in five minutes!” 2020. Available: <https://www.rgdd.se/post/hold-on-to-your-hat-and-learn-system-transparency-in-five-minutes/>
- [29] Keylime, “Keylime: A CNCF project for TPM-based remote attestation.” Available: <https://keylime.dev/>
- [30] IETF SCITT Working Group, “An Architecture for Trustworthy and Transparent Digital Supply Chains.” 2026. Available: <https://datatracker.ietf.org/doc/rfc9943/>